

Quant Guild Recruitment

Trading Bot Competition: Problem Statement

1 Overview

You will program a trading bot that competes against other bots in a multi-round auction. Each round your bot is given a private value and must decide how much to bid. The objective is to maximise profit over the full game across changing market conditions and changing bankrolls.

The competition runs over **four auction variations**, which differ only in how payoffs are assigned once the bids come in. Variations 1 and 2 were released at the orientation session on 09/09; **Variations 3 and 4 are released now, alongside the Mock Auction 1 results.** Everything else in this document, the game mechanics, the observations, the scoring and the tournament structure, is identical across all four.

Bots are submitted on the competition site, quantguildiitm.in, from the /submit page, after signing in with your @smaill.iitm.ac.in Google account.

Please read this document in full. Post any questions on the guild WhatsApp group and we will respond as soon as we can.

2 Timeline

All times IST, dates in DD/MM.

When	What
Wed 09/09, 10:00 pm	Orientation. Variations 1 and 2 released with starter code.
Sat 12/09, 12:00 pm	Deadline for mock submissions of Variations 1 and 2 (optional, strongly recommended).
Sat 12/09, evening	Mock Auction 1 run on Variations 1 and 2; results shared. Variations 3 and 4 released.
Sun 13/09 to Tue 15/09	Mock Auctions 2, 3 and 4 , one each night at EOD, on all four variations. Results shared after every run. Each run uses your latest accepted submission at that moment.
Wed 16/09, 11:59 pm	Final submission deadline , all variations.
After 16/09 EOD	Final evaluation run on the last accepted submission for each variation. Final leaderboards published.

Four mock runs means four chances to see your bot break before it counts. You do not need to re-upload between them — whatever is sitting on the site plays each night — but the whole point is to change something after each one.

3 Game Mechanics

3.1 Structure

- **Players:** $n = 20$ bots per auction group.
- **Rounds:** $t = 2000$, split into **4 blocks of 500 rounds**.
- Each round, every solvent player i independently draws a value x_i from that block's distribution, submits a bid, and receives a payoff determined by the variation in play.
- Capital updates as $\text{new capital} = \text{old capital} + \text{payoff}$.

3.2 How a block is initialised

At the start of every block, the market is re-drawn from scratch. For block b :

1. Draw a **minimum** m_b uniformly from $\{100, 110, 120, \dots, 1000\}$: multiples of 10, so 91 possible values.
2. Draw a **range** R_b uniformly from $\{100, 200, 300, \dots, 1000\}$: multiples of 100, so 10 possible values.
3. For every round in that block, values are drawn as $x_i \sim \mathcal{U}[m_b, m_b + R_b]$, independently for each player and each round.

That gives $91 \times 10 = 910$ possible blocks. **The two grids above are public; the seed is not.** Every iteration draws its own schedule of 4 blocks, and all groups within the same iteration play the same schedule — so no group gets an easier market than another in the same iteration.

Two consequences worth internalising. The minimum is *never* 0: values always sit on a floor of at least 100, and that floor is a large fraction of the value in a narrow block. And a block like $[1000, 1100]$ is exactly as likely as $[100, 1100]$, so knowing the width tells you nothing about the level, and vice versa.

You are never told m_b or R_b . Estimating the distribution from what you observe is a core part of the problem.

The *timing*, on the other hand, you can work out for yourself. Blocks are exactly 500 rounds long and you receive the round index, so boundaries always fall at rounds 501, 1001 and 1501 and you always know which block you are in. The difficulty is not noticing the change. It is that at each boundary everything you had learned about the market becomes worthless at once, and you have to rebuild an estimate from zero samples while still bidding every round.

3.3 Starting capital

Capital is also re-drawn at every block boundary, **independently for each player**. For player i in block b :

$$k_i \sim \mathcal{U}[0.5, 2.5], \quad C_i^{(b,0)} = m_b + k_i \cdot R_b.$$

So within the same block some bots start rich and some start poor, and nobody is told anyone else's capital. Your maximum legal bid is always your current capital. The smallest starting capital the game can produce is 150 ($m_b = 100$, $R_b = 100$, $k_i = 0.5$).

3.4 Capital does not carry over

The capital you finish a block with is banked as score, not as money. It is normalised and converted into leaderboard points (Section 7); the next block then starts from a freshly

drawn capital.

The reason: varying the starting capital is a test of whether your bot adjusts its aggression to its bankroll, $0.5\times$ the range above the floor is a very different situation from $2.5\times$. If capital carried over, that test would break. A bot that drew a large bankroll early and compounded it could stay aggressive for the rest of the game purely because it is rich, so the later blocks would be measuring early luck instead of strategy. Resetting makes each block a clean, independent test, and the leaderboard aggregates four of them per iteration.

Bankruptcy is likewise **per block**: a bot is out for the remainder of the block once its capital reaches 0 or below, and returns at the next block boundary with a fresh capital draw.

3.5 Bids

- Bids may be fractional and must lie in $[0, \text{current capital}]$.
- A bid **above your capital, negative, NaN, infinite, not a number at all, or not returned within the time limit** counts as a bid of 0 for that round. Over-capital bids are **not** clamped down to your capital, they are thrown away entirely, so check your own arithmetic.
- **Ties, Variations 1 and 2:** every bid within 10^{-9} of the top bid wins, and each winner receives the *full* payoff.
- **Ties, Variations 3 and 4:** these variations depend on distinct ranks, so ties are broken **uniformly at random**.

3.6 Edge cases

- A round with fewer than 2 solvent bots is skipped entirely, no values are drawn and no payoffs are assigned.
- In Variations 1 and 2, if every bid is 0, all bots are tied at the top and all of them win.

4 Information available to your bot

At the start of each round your bot receives an observation dictionary with exactly the following keys. A tick means the key is present in that variation.

Description	Key	V1	V2	V3	V4
Round index, 1–2000	<code>round</code>	✓	✓	✓	✓
Your value this round	<code>x</code>	✓	✓	✓	✓
Your current capital	<code>capital</code>	✓	✓	✓	✓
Maximum legal bid (= capital)	<code>max_bid</code>	✓	✓	✓	✓
Solvent bots this round, n_t	<code>num_players</code>	✓	✓	✓	✓
Highest bid last round	<code>highest_bid_last_round</code>	✓	✓	✓	✓
Second-highest bid last round	<code>second_highest_bid_last_round</code>	✓	✓	✓	✓
Your own bid last round	<code>my_last_bid</code>	✓	✓	✓	✓
Your rank last round (1 = won)	<code>my_last_rank</code>	✓	✓	✓	✓
Your payoff last round	<code>my_last_payoff</code>	✓	✓	✓	✓
Realised X of last round	<code>max_value_last_round</code>	—	✓	✓	✓
Top five bids last round, $[b_1 \dots b_5]$	<code>top_bids_last_round</code>	—	—	—	✓

Notes:

- Every “last round” field is 0 in round 1. `top_bids_last_round` is always a list of five floats, zero-padded if fewer than five bids were placed.

- You never learn m_b , R_b , or any other player's value, capital or identity. The block index and the boundary rounds you can compute yourself from **round**.
- n_t matters because in Variations 2–4 the payoff depends on the maximum value across the solvent players. As bots go bankrupt, n_t falls and that maximum falls with it in expectation.
- Your bot may keep internal state across rounds (history buffers, estimators, and so on). State persists for the whole 2000 rounds and is *not* reset at block boundaries, so whatever you learned in the last block is still sitting there at round 501, whether or not it is still true. Clearing it is your job.

5 Auction Variations

Throughout, let $b_1 \geq b_2 \geq b_3 \geq \dots$ be the bids sorted in descending order, and let $X = \max_j x_j$ over the players solvent that round.

1. Variation 1 — private value, first price.

The winner receives $\text{Payoff} = x_i - b_1$, where x_i is the *winner's own* value. Everyone else receives zero.

2. Variation 2 — common value, first price.

The winner receives $\text{Payoff} = X - b_1$. Everyone else receives zero.

3. Variation 3 — runner-up penalty (common value).

Rank 1 receives $X - b_1$. Rank 2 receives $-0.5(X - b_1)$, or 0 when $X - b_1 < 0$ — the runner-up is never charged for the winner's loss. Everyone else receives zero.

4. Variation 4 — funded second price (common value).

If $b_1 \leq X$:

$$\text{Payoff}_{(1)} = X - b_2,$$

$$\text{Payoff}_{(2)} = X - b_1,$$

$$\text{Payoff}_{(k)} = -s_k \cdot T, \quad k \in \{3, 4, 5\},$$

where $T = (X - b_2) + (X - b_1)$ and $s_3 = 0.5$, $s_4 = 0.3$, $s_5 = 0.2$. Ranks 6 and below receive zero, and since the shares sum to 1 the round nets to exactly zero. With fewer than five solvent bots the shares are rescaled over the ranks that do exist; with two solvent bots nobody pays.

If $b_1 > X$: rank 1 alone receives $X - b_1$ (a loss) and every other player receives zero. No penalties are collected.

Reading the four:

- **V1 vs V2:** in Variation 1 you are bidding on something only you can see. In Variation 2 everyone is bidding on the same prize, and winning it is evidence that you overestimated it.
- **V3:** second place is the only losing seat that actually costs money — everyone from rank 3 down walks away flat.
- **V4:** rank 1 pays b_2 , not b_1 , so the prize is cheaper than it looks; the risk sits entirely in the $b_1 > X$ branch, where the winner eats the loss alone. And ranks 3–5 finish *worse off than rank 6*, so there is no safe seat just below the money.

6 Writing your bot

6.1 The shape of a bot

You submit one file per variation, so make one copy of the template for each variation you enter. Use `Template.py` for Variations 1 and 2, `Template_3.py` for Variation 3 and `Template_4.py` for Variation 4.

Each file defines exactly one class called `Bot`, and the engine calls two methods on it:

- `__init__(self, config)` runs once, before round 1. Store whatever you like on `self`. It is identical across all four variations. `config` is a dictionary with `player_id`, `variation`, `num_players`, `num_rounds` and `starting_capital`. Note that `starting_capital` is your *block 1* capital only; later blocks redraw it and you will see the new figure through `obs["capital"]`.
- `get_bid(self, obs)` runs every round. `obs` is a dictionary of everything you know this round. Return a number: your bid.

The only structural difference between the variations is how many keys `obs` contains. Variation 1 gets the fewest, and each later variation gets the same keys or more. Everything else that changes is the payoff rule, which lives in the engine, not in your file.

Every variation receives these:

```
obs["round"]           # 1 to 2000
obs["x"]               # your value this round
obs["capital"]         # your money now, and your max legal bid
obs["num_players"]     # bots still solvent
obs["highest_bid_last_round"]
obs["second_highest_bid_last_round"]
obs["my_last_bid"]
obs["my_last_rank"]    # 1 = you won
obs["my_last_payoff"]
```

6.2 Variation 1: `ROLLNO_1.py`

The common keys, and nothing else. When you win, your payoff is your own x minus your bid.

```
class Bot:
    def __init__(self, config):
        pass

    def get_bid(self, obs):
        bid = 0.5 * obs["x"]
        return max(0.0, min(bid, obs["capital"]))
```

6.3 Variation 2: `ROLLNO_2.py`

You also get `obs["max_value_last_round"]`, which is last round's X , the highest value anyone drew. When you win, your payoff is X minus your bid, so this key is the single most useful thing in the observation.

```

class Bot:
    def __init__(self, config):
        self.past_X = []

    def get_bid(self, obs):
        if obs["round"] > 1:
            self.past_X.append(obs["max_value_last_round"])

        bid = 0.5 * obs["x"]
        return max(0.0, min(bid, obs["capital"]))

```

6.4 Variation 3: ROLLNO_3.py

Exactly the same keys as Variation 2. Only the payoff rule changes: come second and you pay a penalty. Watch `obs["my_last_rank"] == 2` to see when you were charged.

```

class Bot:
    def __init__(self, config):
        self.past_X = []

    def get_bid(self, obs):
        if obs["round"] > 1:
            self.past_X.append(obs["max_value_last_round"])
            if obs["my_last_rank"] == 2:
                pass # you paid a penalty last round

        bid = 0.5 * obs["x"]
        return max(0.0, min(bid, obs["capital"]))

```

6.5 Variation 4: ROLLNO_4.py

Variation 2's keys plus `obs["top_bids_last_round"]`, last round's five highest bids. It is always a list of exactly five floats, zero-padded if fewer than five bids were placed, so unpacking it is safe.

```

class Bot:
    def __init__(self, config):
        self.past_X = []

    def get_bid(self, obs):
        if obs["round"] > 1:
            self.past_X.append(obs["max_value_last_round"])

        b1, b2, b3, b4, b5 = obs["top_bids_last_round"]

        bid = 0.5 * obs["x"]
        return max(0.0, min(bid, obs["capital"]))

```

6.6 In short

File	Extra keys available
ROLLNO_1.py	none
ROLLNO_2.py	max_value_last_round
ROLLNO_3.py	max_value_last_round
ROLLNO_4.py	max_value_last_round, top_bids_last_round

Two rules that apply everywhere:

- **Never read a key your variation does not get.** Reaching for `top_bids_last_round` in a Variation 2 file raises an error, and an errored round is scored as a bid of 0. Do it every round and your upload fails the 30-errors-in-120-rounds check outright.
- **Always finish with** `return max(0.0, min(bid, obs["capital"]))`. The engine does not clamp an over-capital bid down to your capital, it throws the bid away and scores you at 0, so the clamp has to be yours.

The `0.5 * obs["x"]` in every skeleton above is one of the published sample bots. It is deliberately bad. Replacing it is the entire competition.

7 Scoring

Raw profit is not comparable across blocks — a block with $M_b = 2000$ produces payoffs an order of magnitude larger than one with $M_b = 200$, and starting capitals differ between players inside the same block. Profit is therefore normalised in two steps. The full specification is at quantguilder.in/api/docs/scoring.pdf.

Step 1 — scale. Write $M_b = m_b + R_b$ for the block's maximum possible value. For player i in block b ,

$$\pi_i^{(b)} = \frac{C_i^{(b,\text{end})} - C_i^{(b,0)}}{M_b},$$

i.e. profit measured in units of that block's ceiling. A bankrupt bot has $C^{(b,\text{end})} = 0$, giving $\pi_i^{(b)} = -C_i^{(b,0)}/M_b$.

Step 2 — standardise within the group. Let μ_b and σ_b be the population mean and standard deviation of $\pi^{(b)}$ over the bots in your group for that block. Then

$$z_i^{(b)} = \text{clip}\left(\frac{\pi_i^{(b)} - \mu_b}{\sigma_b}, -3, 3\right), \quad P_i^{(b)} = 50 + 15 z_i^{(b)} \in [5, 95].$$

Clipping at ± 3 prevents one freak block from deciding the competition. If $\sigma_b = 0$, every bot in that block scores exactly 50.

$$\text{Iteration score} = \sum_{b=1}^4 P_i^{(b)}, \quad \text{Total} = \sum_{\text{iterations}} \text{Iteration score}.$$

Note that the divisor in Step 1 is a positive constant within a block, so it cancels out of the z -score entirely: the points P would be identical if we divided by R_b instead. The choice of M_b only affects the reported π -based diagnostics below, which are stated in units of the block ceiling.

Published per bot: total score, mean block points, mean π , spread (population SD of π across blocks), worst-block π , survival rate, raw net-profit statistics, wins, errors and timeouts. Disqualified bots are ranked last.

8 Tournament Structure

A single 20-bot arrangement is far too noisy to rank on — who you are grouped with matters nearly as much as how you play. Each variation is therefore run over **5 iterations**, each with its own seed and its own 4-block schedule. Seeds are published afterwards so results are reproducible.

- **Iterations 1 and 2: random.** Bots are shuffled into groups under two independent seeds. Full 2000 rounds, 4 blocks each.
- **Iteration 3: strength-balanced.** Cumulative points from iterations 1–2 are used to *balance* the groups, not to segregate them: bots are sorted by points and dealt into groups in a snake pattern, so every group has similar average strength. This sharply reduces the variance of the standardised score while keeping cross-group comparison fair.
- **Iterations 4 and 5: finals.** The top 20 bots by cumulative score after iteration 3 play two head-to-head iterations under fresh seeds.

Final ranking is the sum of scores over all iterations a bot played, tie-broken by mean block points. In the final evaluation run, there could be more than 5 iterations to evaluate more concretely.

8.1 Grouping

The field is split into $\lceil n/20 \rceil$ groups of as near to equal size as possible — 54 bots become 18/18/18, never 20/20/14. Any group short of 20 is filled up with the published sample bots. Fillers play for real and count in the standardisation of that group, but they never appear on the leaderboard. The random and balanced iterations use the same split; only the order in which bots are dealt into it differs.

8.2 Mock auctions

The mocks run the **same full 5-iteration structure** as the real thing. There are four of them: Mock Auction 1 on 12/09 (Variations 1 and 2 only), then one every night at EOD on 13/09, 14/09 and 15/09 across all four variations. None of them count toward the final result. There is no live leaderboard between rounds — only the mock boards and the final board are published. After every run, each participant receives a per-block table (C_{start} , C_{end} , M_b , π , P , bankrupt) showing exactly where their bot broke.

9 Sample Round

Variations 1–3

Three bots, capital 500 each. Suppose the hidden block has $m_b = 200$, $R_b = 300$, so values fall in $[200, 500]$.

- Bot 1 — always bids 15 more than its own value.

- Bot 2 — always bids 5 more than its own value.
- Bot 3 — always bids half its own value.

The values drawn are 230, 250, 260, so $X = 260$. The bids are 245, 255, 130. **Bot 2 wins; Bot 1 is rank 2.**

	Variation 1	Variation 2	Variation 3
Bot 2 (rank 1)	$250 - 255 = -5$	$260 - 255 = +5$	$260 - 255 = +5$
Bot 1 (rank 2)	0	0	$-0.5 \times 5 = -2.5$
Capital after	500 / 495 / 500	500 / 505 / 500	497.5 / 505 / 500

Variation 4

Six bots, capital 500 each, same block [200, 500].

	Value	Bid	Rank	Payoff	Capital after
Bot 6	255	258	1	$X - b_2 = 260 - 255 = +5$	505
Bot 2	250	255	2	$X - b_1 = 260 - 258 = +2$	502
Bot 5	245	250	3	$-0.5 \times 7 = -3.5$	496.5
Bot 1	230	245	4	$-0.3 \times 7 = -2.1$	497.9
Bot 4	220	240	5	$-0.2 \times 7 = -1.4$	498.6
Bot 3	260	130	6	0	500

Here $X = 260$ and $b_1 = 258 \leq X$, so the funded branch applies. The top two earn $T = 5 + 2 = 7$ between them, and ranks 3–5 pay exactly that 7 back. Note that Bot 3 bid the *least* of anyone and still ends the round better off than Bots 1, 4 and 5.

Had Bot 6 bid $262 > X$ instead, it alone would have received $260 - 262 = -2$ and no other bot would have been affected at all.

10 Deliverables

Submit one Python file per variation, plus a **short report** explaining your strategy.

- **Naming:** ROLLNO_1.py through ROLLNO_4.py, matching the variation — e.g. 0B24C420_3.py. Roll numbers are two letters, two digits, a letter, then three digits.
- You may enter any subset of the variations. **Each variation has its own leaderboard;** there is no combined score across variations.
- **Upload at** /submit on quantguildiitm.in, signed in with your gmail account. The roll number in the filename must match your gmail account.
- Every upload is checked immediately: (1) filename, (2) a code policy check, (3) a 120-round test game against the sample bots. It is rejected if it crashes, goes bankrupt, or errors on more than 30 of the 120 rounds.
- You may resubmit as often as you like, with a 60-second cooldown between uploads. **The latest accepted file for each variation is the one that plays.** Only variations that have been released are accepted.

Your report should cover, for each variation: the core idea, how you estimate the hidden distribution, how your bid depends on your starting capital, how you detect and react to a block change, and which libraries you used.

The report goes through the submission form, not the site:

<https://forms.gle/eDAQ47LsaDsvbeiM9>

Final deadline: Wednesday 16th September 2026, 11:59 pm IST.

11 Rules and Constraints

Starter kit

- `Template.py` (Variations 1 and 2), `Template_3.py`, `Template_4.py` — edit only these.
- `local_test.py` — a rough local stand-in that uses **made-up bounds and only 3–4 bots**. It is *not* the real engine, so treat its numbers as a smoke test and run it over several seeds before you trust anything.
- `sample_bots/` — the three published bots ($x+15$, $x+5$, $0.5x$), `README.md` and `README_v3_v4.md`.
- The engine source is not included.

Limits

- Under **1 second per round**. A round that times out is scored as a bid of 0.
- Under **100 MB** of memory.

Allowed libraries

Only the following imports are permitted: `math`, `cmath`, `random`, `statistics`, `decimal`, `fractions`, `itertools`, `functools`, `operator`, `collections`, `heapq`, `bisect`, `array`, `copy`, `enum`, `dataclasses`, `typing`, `numbers`, `abc`, `string`, `re`, `json`, `time`, `datetime`, `warnings`, `numpy`, `pandas`, `scipy`, `sklearn`, `statsmodels`.

List the ones you actually use in your report.

Prohibited

- `eval`, `exec`, `compile`, `open`, `__import__`, `input`, and dunder attribute access are rejected at upload time.
- Any file, process or network access, or any attempt to inspect the simulator, means **disqualification**.

12 What We Are Looking For

- **The report and the reasoning behind your strategy** — this is the primary metric.
- Net profit alone is not a metric. It is read alongside how the strategy adapts.
- **Robustness to starting capital**: does aggression genuinely respond to bankroll, or is it fixed?
- **Robustness across blocks**: consistency of π across the four regimes, and how fast the bot re-adapts after a boundary.
- **Survival**: bankruptcy is heavily penalised — it forfeits the rest of the block.



13 Resources

- Investopedia — Random Variables
- W3Schools — Python Introduction
- W3Schools — NumPy Introduction

Feel free to make extensive use of AI — we are testing the logic behind your strategy, not your coding skills. That said, please indent properly and comment where it helps us read your code.

ALL THE BEST!